Agent Trajectory Failure Detection: A Benchmark for Evaluating Agent Monitoring Tools

Sohan Shingade
Galea
sohan@galea.foo

May 2026

Abstract

As AI agent workflows become production business processes, tools that monitor these workflows must detect failures automatically—without requiring users to write evaluation rules. We introduce **Agent Trajectory Failure Detection (ATFD)**, a benchmark task that measures how well a monitoring tool can (1) detect failed agent trajectories, (2) avoid false positives on successful trajectories, and (3) correctly categorize the type of failure. We evaluate on 150 trajectories from τ-bench [Sierra Research, 2025], spanning retail and airline customer-service domains. We report results for Galea, a heuristic-based agent investigation system, achieving 94.4% detection rate with 0% false positive rate and 79% failure-type alignment—using zero user-configured evaluation rules. We release the benchmark methodology, converter, and evaluation harness to enable comparison across agent monitoring tools.

1 Introduction

Agent observability tools—LangSmith, Langfuse, Arize Phoenix, Braintrust—provide trace collection and visualization for LLM-based agent workflows. However, none of these tools automatically detect whether a given agent trajectory represents a failure. Users must write custom evaluation rules, set alert thresholds, or manually review traces.

This gap matters as agent workflows increasingly handle consequential business tasks: processing refunds, modifying reservations, exchanging products, and handling customer complaints. A customer-service agent that calls the wrong tool, passes incorrect arguments, or enters a tool-call loop represents a real business failure—not a model quality issue.

Existing agent benchmarks—τ-bench [Sierra Research, 2025], AgentBench [Liu et al., 2024], AgentBoard [Ma et al., 2024], SWE-bench [Jimenez et al., 2024]—evaluate *agents* on task completion. No benchmark evaluates the *tools that monitor agents*. We address this gap.

We propose Agent Trajectory Failure Detection (ATFD) as a benchmark task. The task is simple: given a complete agent trajectory (the full sequence of messages, tool calls, and tool results from a multi-turn interaction), determine whether the trajectory represents a failure, and if so, what kind.

2 Task Definition

2.1 Input

A complete agent trajectory consisting of:

- System prompt and user messages (customer requests)
- Assistant messages (agent responses)
- Tool calls (name, arguments, requestor)
- Tool results (output, error status)
- Termination reason

2.2 Output

A set of **findings**, each with:

- **Severity:** error, warning, or info
- **Category:** failure type (e.g., wrong action, tool loop, tool risk, policy violation)
- **Attribution:** which agent or tool is responsible (optional)

2.3 Ground Truth

Provided by τ-bench's evaluation framework:

- `reward` (0.0 = complete failure, 1.0 = success)
- `reward_breakdown` (per-component: DB state match, action correctness, communication completeness)
- `termination_reason` (normal stop, error, timeout, max steps)

3 Metrics

3.1 Detection Rate

Percentage of failed trajectories (reward < 1.0) where the tool produced at least one substantive finding:

$$\text{Detection Rate} = \frac{|\{t \in T_{\text{fail}} : \text{findings}(t) \neq \emptyset\}|}{|T_{\text{fail}}|}$$

3.2 False Positive Rate

Percentage of successful trajectories (reward $= 1.0$) where the tool produced error-severity findings:

$$\text{FP Rate} = \frac{|\{t \in T_{\text{pass}} : \text{error\_findings}(t) \neq \emptyset\}|}{|T_{\text{pass}}|}$$

Baseline anomaly findings (cost/latency/volume deviations from project median) are excluded from the FP calculation, as these are informational and expected when the project baseline has few samples.

3.3 Category Alignment

Percentage of τ-bench failure types correctly covered by the tool's finding categories. The mapping between τ-bench failure types and tool finding categories is defined in Table 1.

3.4 Configuration Effort

Number of evaluation rules, scorers, or alert thresholds the user must configure before the tool produces findings. Lower is better. Zero means fully automatic detection.

Table 1: Category mapping between τ-bench failure types and tool finding categories.

τ-bench failure type	Matching tool categories
`wrong_action`	tool_error, tool_risk_halt, tool_risk_warn, loop_suspect
`wrong_state_change`	tool_risk_halt, tool_risk_warn, failure
`wrong_communication`	correctness_no_citation, hallucination
`missing_information`	correctness_no_citation
`env_violation`	policy_deny, tool_risk_halt

4 Dataset

We use pre-recorded simulation results from τ-bench [Sierra Research, 2025], a benchmark for evaluating LLM agents on customer-service tasks.

Domains. Retail (100 trajectories): product exchanges, order cancellations, address modifications. Airline (50 trajectories): reservation changes, cancellations, certificate issuance.

Agent configuration. GPT-4.1 (2025-04-14) with default τ-bench scaffold. User simulator: GPT-4.1 (2025-04-14). Four trials per task.

Trajectory size. Each trajectory contains 10–53 trace events after conversion, including user messages, model calls, tool invocations, and tool results.

Table 2: Failure distribution across domains.

Domain	Total	Failed ($r < 1.0$)	Pass rate
Retail	100	22	78%
Airline	50	14	72%
Combined	150	36	76%

5 Evaluated System: Galea

Galea is an investigation layer for agent workflows. Its heuristic investigator operates without LLM calls and detects failures through five mechanisms:

1. **Event pattern matching**—run failures, policy denials, approval rejections, tool errors
2. **Counting**—tool-call repetition ($\geq$4 calls to the same tool flags a loop suspect)
3. **Statistical comparison**—current trace vs. project baseline median (2× = warning, 5× = error)
4. **Cross-referencing**—cited documents vs. indexed documents in the data room
5. **Risk scoring**—tool permission level × repetition × resource sensitivity

No LLM calls, no embeddings, no user-configured rules. The system ingests trace events and produces findings automatically.

Conversion. τ-bench `SimulationRun` objects are converted to Galea's normalized `TraceEvent` format. Each message becomes the appropriate event type (`signal_received`, `model_called`, `tool_called`, `tool_completed`, `tool_failed`). The converter is included in the open-source release.

6 Results

Table 3: ATFD benchmark results for Galea (heuristic investigator, zero configuration).

Metric	Retail (n=100)	Airline (n=50)	Combined
Detection Rate	90.9%	100.0%	94.4%
False Positive Rate	0.0%	0.0%	0.0%
Category Alignment	76.2%	85.0%	79.0%
Avg. findings (failed)	3.4	4.3	3.8
Avg. findings (passed)	2.7	1.7	2.3
Config effort	0	0	0

6.1 Failure Analysis

The two undetected failures in the retail domain were trajectories where the agent called the correct tools in the correct order but passed **wrong arguments** (e.g., exchanging item A for item B instead of item A for item C). The trace shape was identical to successful trajectories—only the argument values differed. Detecting these requires comparing tool arguments against expected outcomes or using LLM-backed semantic analysis, which the heuristic layer does not perform.

Galea includes an optional LLM-backed investigation layer (not evaluated here) that compares agent outputs against a mission statement derived from the input signal and project configuration. We expect this layer to close the gap on argument-level correctness failures.

6.2 Finding Distribution

Most common finding categories on failed trajectories:

- `tool_risk_warn`—state-changing tools flagged for review
- `loop_suspect`—repeated tool calls indicating planning failures
- `anomaly_volume`—more events than project baseline

The airline domain achieved higher category alignment (85% vs. 76.2%) because airline tasks involve more state-changing operations (reservation modifications, cancellations), which the tool-risk scorer surfaces effectively.

7 Comparison with Existing Tools

No existing agent observability tool provides automatic trajectory-level failure detection without user configuration.

This benchmark is tool-agnostic. We invite submissions from any agent monitoring tool. Tools that require configuration should report what was configured (number of rules, setup time) alongside detection metrics.

Table 4: Comparison of agent monitoring tools on automatic failure detection capability.

Tool	Auto-detection	Configuration required
Galea	**Yes (heuristic)**	**None**
LangSmith	No	User writes eval rules
Langfuse	No	User defines scores
Arize Phoenix	No	User configures evaluators
Braintrust	No	User writes scorers
DeepEval	No	User selects metrics

8 Reproducing Results

The benchmark harness, converter, and evaluation code are available at:

`https://github.com/Galea-foo/Galea/tree/main/benchmarks/tau-bench`

9 Conclusion

We introduce Agent Trajectory Failure Detection (ATFD), the first benchmark for evaluating agent monitoring tools—as opposed to agents themselves. On 150 τ-bench trajectories, Galea's heuristic investigator achieves 94.4% detection rate with 0% false positive rate, using zero user-configured rules. We release the benchmark to enable systematic comparison across the growing ecosystem of agent observability and investigation tools.

References

Sierra Research. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. `https://github.com/sierra-research/tau2-bench`, 2025.

Xiao Liu, Hao Yu, Hanchen Zhang, et al. AgentBench: Evaluating LLMs as Agents. In *ICLR*, 2024.

Chang Ma, Junlei Zhang, Zhihao Zhu, et al. AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents. In *NeurIPS*, 2024.

Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR*, 2024.

Yifan Yao, Yiran Wu, et al. Beyond Black-Box Benchmarking: Observability, Analytics, and Optimization of Agentic Systems. In *KDD*, 2025.